BLoC

Cubit, events, and one test that proves the state

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Week 10

TODAY

What this lab is for

Separate logic from UI

Move the todo list out of setState and into a Cubit, then a Bloc, while the screen only reacts to state.

Prove it with a test

Write one bloc_test that drives events through the bloc and checks the exact states it emits, in order.

TIME

2 hours

SUBMIT

Push to your admd-labs repo, branch `lab-5`, before next lab

SETUP

Where your code goes

1. Open the project from Lab 4 and create a branch: `git checkout -b lab-5`
2. Keep `lib/models/todo.dart` from Lab 4. Create a new `lib/todos/` folder for everything else.
3. Run the app once before changing anything. If it does not build, fix that first.

Hint

Hot restart after wrapping the app in `BlocProvider`: a hot reload can leave a stale provider in the tree.

SETUP

The feature’s folder structure

```
lib/
  models/todo.dart
  todos/
    todo_repository.dart
    cubit/todo_cubit.dart
    cubit/todo_state.dart
    bloc/todo_bloc.dart
    bloc/todo_event.dart
    bloc/todo_state.dart
  screens/todo_screen.dart
  main.dart
test/todos/todo_bloc_test.dart
```

SETUP

Packages to add

```
flutter pub add flutter_bloc equatable
flutter pub add --dev bloc_test mocktail
```

1. `flutter_bloc`: Cubit, Bloc, and the widgets that read them.
2. `equatable`: value equality for state classes.
3. `bloc_test`: drives a bloc and asserts what it emits.
4. `mocktail`: fakes the repository in the test.

Hint

`flutter pub add` writes the resolved version into `pubspec.yaml`. Commit that change together with your code.

TASK I

TodoCubit: state without events

1. Create `TodoState extends Equatable` holding the todo list and a loading flag.
2. Create `TodoCubit extends Cubit<TodoState>` with `load`, `add`, `toggle`, `remove`.
3. Wrap the app in `BlocProvider` whose `create` returns `TodoCubit()..load()`.
4. Replace every `setState` in `TodoScreen` with a `BlocBuilder`.

DONE WHEN

- [ ] No `setState` remains in `TodoScreen`.
- [ ] Toggling calls `TodoCubit`'s `toggle(id)` via `context.read`.
- [ ] Two states with the same todos compare equal.

A cubit method and a bloc event both end in emit. Task II only changes how the call arrives, not what happens after it.

TASK I

The cubit’s state

```
class TodoState extends Equatable {
  final List<Todo> todos;
  final bool loading;
  const TodoState({this.todos = const [], this.loading = false});
  TodoState copyWith({List<Todo>? todos, bool? loading}) => TodoState(
        todos: todos ?? this.todos,
        loading: loading ?? this.loading,
      );
  @override
  List<Object?> get props => [todos, loading];
}
```

TASK I

TodoCubit: four methods, one shape

```
class TodoCubit extends Cubit<TodoState> {
  TodoCubit() : super(const TodoState());
  Future<void> load() async {
    emit(state.copyWith(loading: true));
    emit(TodoState(todos: await fetchTodos()));
  }
  void toggle(int id) => emit(state.copyWith(todos: [
        for (final t in state.todos)
          if (t.id == id) t.copyWith(completed: !t.completed) else t,
      ]));
}
```

TASK I

Wiring: BlocProvider and BlocBuilder

```
void main() => runApp(BlocProvider(
      create: (_) => TodoCubit()..load(),
      child: const TodoApp(),
    ));
// inside TodoScreen.build
BlocBuilder<TodoCubit, TodoState>(
  builder: (context, state) => state.loading
      ? const CircularProgressIndicator()
      : TodoList(
          todos: state.todos,
          onToggle: context.read<TodoCubit>().toggle,
        ),
)
```

REFERENCE

From Cubit to Bloc: why promote

	CUBIT	BLOC
Trigger	a direct method call	an event, via `add()`
Multiple sources	harder to compose	one `on<Event>` per source
Test grain	one call per case	one event per case
Talks to the API	directly, or not at all	through a `TodoRepository`
This lab	Task I	Task II onward

Promote when one state can change for more than one reason. Naming each reason as an event keeps the bloc's history readable and testable.

TASK II

TodoBloc: events and a repository

1. Define `TodoEvent` subclasses: `LoadTodos`, `AddTodo`, `ToggleTodo`, `RemoveTodo`.
2. Wrap Lab 4's `fetchTodos()` inside a `TodoRepository` class.
3. Create `TodoBloc`, a `Bloc<TodoEvent, TodoState>`, with one `on<Event>` handler per event.
4. Swap `TodoCubit` for `TodoBloc` in `BlocProvider` and dispatch with `.add(...)`.

DONE WHEN

- [ ] Every action calls `.add(...)`, never a cubit method.
- [ ] Only `TodoRepository` calls `dio`.
- [ ] Toggling and removing update the visible list.

The repository is the only class that talks to the network. TodoBloc receives todos; it never builds a Dio client itself.

TASK II

The event hierarchy

```
sealed class TodoEvent {}

class LoadTodos extends TodoEvent {}

class AddTodo extends TodoEvent {
  AddTodo(this.title);
  final String title;
}

// ToggleTodo(id) and RemoveTodo(id) carry the
// same single int id field as AddTodo carries title.
```

TASK II

TodoRepository: the only class that calls dio

```
class TodoRepository {
  TodoRepository(this._dio);
  final Dio _dio;
  static const _url = 'https://jsonplaceholder.typicode.com/todos';

  Future<List<Todo>> fetchTodos() async {
    final res = await _dio.get(_url);
    return (res.data as List)
        .map((json) => Todo.fromJson(json))
        .toList();
  }
}
```

TASK II

TodoBloc: one handler per event

```
class TodoBloc extends Bloc<TodoEvent, TodoState> {
  TodoBloc(this._repo) : super(const TodoState()) {
    on<LoadTodos>(_onLoad);
  }
  final TodoRepository _repo;
  Future<void> _onLoad(LoadTodos e, Emitter<TodoState> emit) async {
    emit(TodoState(todos: await _repo.fetchTodos()));
  }
}
```

Hint

`on<ToggleTodo>`, `on<AddTodo>` and `on<RemoveTodo>` register the same way.

TASK III

Proving it with bloc_test

1. Add `test/todos/todo_bloc_test.dart`.
2. Fake the repository: extend `Mock` and implement `TodoRepository`.
3. Use `blocTest` to add a todo, then toggle it, and list both states in `expect`.
4. Run `flutter test` and confirm it is green.

DONE WHEN

- [ ] The test builds `TodoBloc` with the fake, never the real repository.
- [ ] `act` fires `AddTodo` then `ToggleTodo`, in that order.
- [ ] `expect` lists both intermediate states, not just the last.

TASK III

One test, two emitted states

```
void main() {
  final repo = FakeTodoRepository();
  blocTest<TodoBloc, TodoState>(
    'add then toggle emits two states in order',
    build: () => TodoBloc(repo),
    act: (bloc) => bloc
      ..add(AddTodo('Milk'))
      ..add(ToggleTodo(1)),
    expect: () => [
      TodoState(todos: [Todo(id: 1, title: 'Milk')]),
      TodoState(todos: [Todo(id: 1, title: 'Milk', completed: true)]),
    ],
  );
}
```

TROUBLESHOOTING

Errors you will see, and the fix

`Bad state: Cannot emit new states after calling close`	An async call finished after the bloc closed. Guard it, or cancel it in `close()`.
`BlocProvider.of() called with a context that does not contain a TodoBloc`	The reading widget sits above the BlocProvider. Move BlocProvider up the tree.
`MissingStubError: 'fetchTodos'`	The mocktail fake was never stubbed with `when(...)` before the test called it.
`Toggling a todo does nothing on screen`	The new state is `==` to the old one. Check that `copyWith` swaps the toggled item.

Push before you leave.

Branch lab-5 · TodoBloc replaces TodoCubit · one passing bloc_test in the PR